Connections and Processing Between Users and Data

1. Overview

After clarifying requirements and defining APIs, the next step is:

> Convert requirements into architecture diagrams showing how users, systems, and data interact.

This is where design becomes concrete.

2. Why This Matters in Interviews

Interviewers want to see:

- Can you structure systems visually?
- Can you break complex systems into layers?
- Can you reason about data movement?
- Can you identify bottlenecks/failures?

3. Multi-Phase Design Thinking Framework

Phase 1: High-Level Actors + Systems

Start with boxes.

Draw:

Users

- Consumer user
- Admin
- Partner service
- Internal employee

Systems

- Web App
- Mobile App
- Backend
- Database

Example

```
None
[User] → [Web App] → [Backend] → [DB]
```

Goal of Phase 1

Clarify:

- Who uses system?
- Which systems exist?
- Who talks to whom?

Phase 2: Separate Processing & Storage

Now split responsibilities.

Instead of:

```
None
Backend
```

Convert into:

```
None
API Layer

Business Logic

Cache

Database

Queue

Analytics
```

Example

```
User → API Gateway → App Service
                        ↓
                      Cache
                        ↓
                       DB
```

Real-Time vs Eventual Consistency

At this stage choose architecture based on requirements.

Real-Time Example

Chat App:

```
User A → WebSocket Server → User B
```

Need:

- Low latency
- Immediate delivery

Eventual Consistency Example

Analytics:

```
None
User Action → Queue → Worker → Data Warehouse
```

Need:

- High throughput
- Async acceptable

Phase 3: Break into Components

Now decompose services.

Example: Backend Service

Split into:

- Auth Service
- User Service
- Feed Service
- Notification Service

```
None
API Gateway

   ↓

 ┌──────────────────┐

 |Auth Service     |

 |Feed Service     |

 |User Service     |

 └──────────────────┘
```

Why This Matters

Shows:

- Modularity
- Team ownership
- Independent scaling

Phase 4: Production Readiness

Now discuss real engineering concerns.

Logging

- Request logs
- Error logs

Monitoring

- CPU
- Memory
- Latency
- Error %

Alerting

- Pager alerts
- SLA breaches

Security

- TLS

- JWT
- Encryption at rest
- RBAC

Fault Tolerance

Ask for each component:

- What if cache fails?
- What if DB replica lags?
- What if queue backs up?

4. Strong Interview Progression

Start broad:

```
None
Users → App → DB
```

Then zoom in:

```
None
Users → LB → API → Services → Cache → DB → Queue
```

Then deep dive:

```
None
Feed Service → Fanout Workers → Redis → Cassandra
```

5. Shared Services (Important)

Common reusable services:

- Auth
- Notifications
- Search
- Logging platform
- Metrics platform
- CDN

Mentioning shared services = strong maturity signal.

6. Data Flow Thinking

Always explain:

Read Path

```
User → API → Cache → DB
```

Write Path

```
User → API → DB → Queue → Cache Invalidate
```

7. Failure Analysis Framework

For every component ask:

Component	Failure	Mitigation
Load Balancer	Down	Multi-AZ LB
Cache	Miss storm	Fallback DB
DB	Crash	Replicas
Queue	Delay	Retry/DLQ
Service	Crash	Auto restart

8. Interview Script

Say:

> “I’ll first draw the user actors and core systems, then separate compute from storage, then decompose services, and finally discuss reliability, monitoring, and security.”

This sounds senior-level.

9. C4 Model (Very Valuable Mention)

Levels:

1. Context → Users + system
2. Container → Apps + DB + services
3. Component → Internal modules
4. Code → Classes/functions

Mentioning this impresses interviewers.

10. Example: Instagram Feed

None

```
Users
 ↓
CDN
 ↓
LB
 ↓
Feed Service
 ↓
Redis Cache
 ↓
Post DB
 ↓
Fanout Queue
```

11. Final Summary

- Use diagrams to convert requirements into architecture
- Start with actors + systems
- Separate processing and storage
- Break systems into components
- Add observability + security
- Analyze failures

What Interviewers Secretly Want

Not art.

They want structured thinking.